## EP5. PDHG and cuPDLP for large-scale linear programs

Lidivine Kengne | IE 5311-001 (D01), Homework 5

**1. Why a first-order method for LP at all.** Both mature methods solve linear systems: simplex factorizes the basis $A_B$, interior point factors $A\Theta A^\top$ every iteration. That is where the memory goes, because even for sparse $A$ the factors fill in and the fill is not under the modeller's control. Past hundreds of millions of nonzeros they stop fitting, and the solver fails for reasons unrelated to the iterations it needed.

**2. The main idea of PDHG.** Take $\min c^\top x$ subject to $Ax \geq b$, $x \geq 0$. Its Lagrangian, with $y \geq 0$ on the rows, is $L(x, y) = c^\top x + y^\top(b - Ax)$, and strong duality makes the LP equivalent to the saddle point problem $\min_{x\geq 0} \max_{y\geq 0} L(x, y)$. This is the Lagrangian derived in Problem 2 Part 5; the difference is that there $x$ was minimised out to leave a dual, whereas here *neither* side is eliminated. Primal–dual hybrid gradient takes one projected gradient step in each variable, extrapolating the primal:

$$x^{k+1} = \mathrm{proj}_{x\geq 0}\big(x^k - \tau(c - A^\top y^k)\big), \quad y^{k+1} = \mathrm{proj}_{y\geq 0}\big(y^k + \sigma\big(b - A(2x^{k+1} - x^k)\big)\big).$$

Projection onto an orthant or a box is a componentwise max, so an iteration costs exactly one $Ax$ and one $A^\top y$: $O(\mathrm{nnz}(A))$ work and $O(\mathrm{nnz}(A))$ memory, no fill-in ever, no linear system anywhere. Convergence needs $\tau\sigma\|A\|_2^2 < 1$ and the averaged iterates close the primal–dual gap at rate $O(1/k)$. The term $2x^{k+1} - x^k$ is not cosmetic: without it this is Arrow–Hurwicz, which need not converge.

**3. What the plain iteration actually does.** Run on the five random instances of Problem 3, against the revised simplex of Part 2, counting iterations to drive primal residual, dual residual and relative gap all below a tolerance (`ep5_pdhg.py`):

| $n$ | $m$ | simplex | plain $10^{-4}$ | plain $10^{-6}$ | rs. $10^{-4}$ | rs. $10^{-6}$ | rs. $10^{-8}$ |
|---|---|---|---|---|---|---|---|
| 5 | 3 | 1 | 15 400 | — | 250 | 375 | 525 |
| 20 | 10 | 9 | 53 775 | — | 1 075 | 2 050 | 3 025 |
| 50 | 20 | 20 | 241 900 | — | 7 125 | 47 350 | 90 850 |
| 100 | 40 | 31 | — | — | 15 200 | 42 100 | 83 625 |

A dash is "not reached in 400 000 iterations". Plain PDHG never got six digits on *any* instance, and on the largest never got four. Restarting here is the crudest version — average the iterates and restart from the average every 100 steps — and it buys two to three orders of magnitude. The engineering below is therefore not optional.

*Restarting* is the largest: on an LP the normalised duality gap is *sharp*, so restarting from the running average converts the $O(1/k)$ rate into a linear one, which is the effect the table above measures. A *primal weight* splits the step as $\tau = \eta/\omega$, $\sigma = \eta\omega$ and adapts $\omega$ so primal and dual residuals fall together. An *adaptive step* replaces the global, pessimistic bound $\eta < 1/\|A\|_2$ by one accepted or rejected on local curvature. *Presolve and Ruiz scaling* come first in execution and last in glamour: conditioning of $A$ sets the constant in every bound above.

**4. PDLP, and how cuPDLP applies it at scale.** cuPDLP is PDLP moved onto a GPU, and the reason it works is that the iteration was already the right shape. Each step is two sparse matrix–vector products plus componentwise arithmetic and a few reductions for the restart and step-size tests — all data-parallel, all bandwidth-bound, which is precisely what GPU memory systems deliver. Nothing in the method is sequential: there is no pivot order to respect, no factorization to build, no triangular solve. That is the contrast with the alternatives, since a simplex pivot sequence is inherently serial and a sparse Cholesky parallelises only within its elimination tree while filling in badly. Memory tells the same story: the method holds $A$ in sparse form plus a handful of vectors of length $n$ and $m$, so instances whose *factors* would exhaust host memory have a *footprint* that fits on a single device. Large and sparse is therefore the regime where the method is strongest, and why first-order LP has since appeared in commercial solvers.

**5. What it costs.** The table prices the first cost: accuracy is bought by iteration count, where simplex returns an exact vertex in 31 pivots. There is also no basis, so no warm start for branch and bound and no clean sensitivity analysis — the duals come out approximate, which matters for exactly the shadow-price reading done in Homework 1. At this size simplex is the right answer; the case for PDHG begins where the factors stop fitting.